

Yes. I reviewed the uploaded voicecartAI-main.zip and re-evaluated it against everything we discussed: phone-call-first ordering, cloud/API AI services, real restaurant deployment, multi-tenant SaaS, production security, horizontal scalability, and the later possibility of swapping cloud AI for local/open-source inference. The roadmap below is therefore based on your actual MVP architecture, not a generic greenfield architecture.

The most important architectural decision is this:

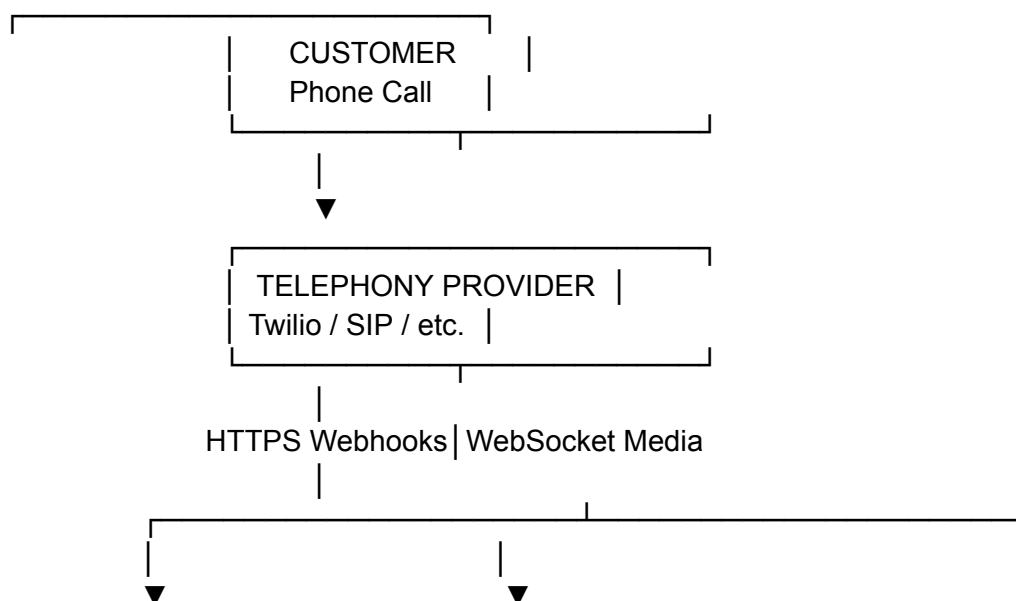
> Do not turn the MVP into 12 microservices immediately.

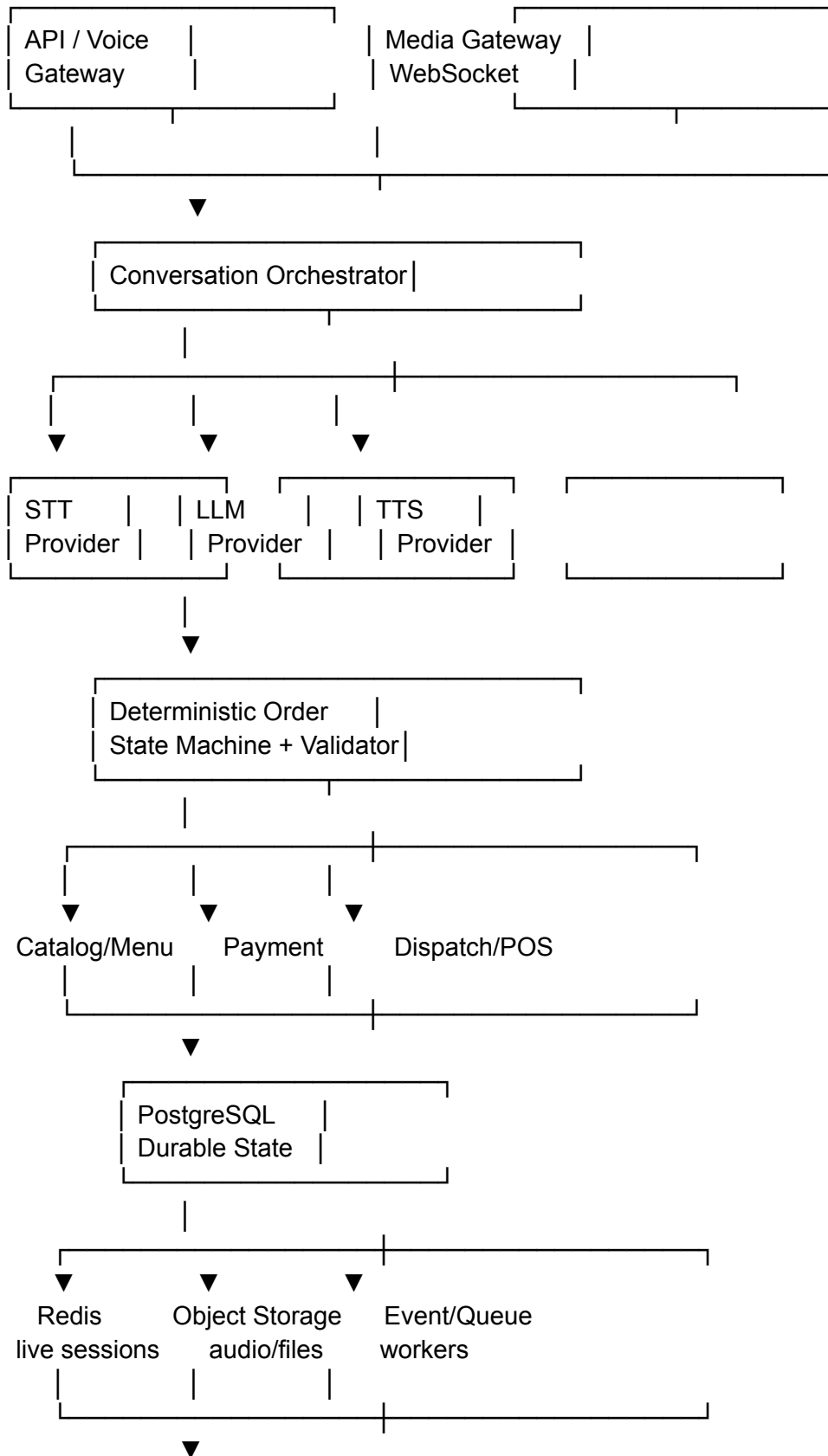
For a solo founder, the production target should initially be a modular monolith + dedicated worker process + managed infrastructure, with clean service boundaries inside the code. That gives you horizontal scalability without creating a distributed-systems burden before you have customers.

Your current application is already a useful prototype, but the main production issues are structural: server.js is doing routing, WebSocket handling, session management and orchestration; sessions live in memory; SQLite is used as the primary database; recordings are written to the local filesystem; routes have no authentication/tenant boundary; WebSockets are not authenticated; external services can silently fall back to mock behavior; and the frontend directly owns API connectivity and polling logic. Those are the areas to fix first.

## 1. Target production architecture

Use this as the final architectural model.





|                       |  |
|-----------------------|--|
| Merchant Dashboard    |  |
| React Web Application |  |

The important distinction is:

AI = interpretation

Backend = authority

Database = source of truth

Queue = reliable side effects

Redis = ephemeral state

Object storage = files

Frontend = presentation

That should govern every refactor.

---

2. First principle: separate the control plane from the voice plane

Your phone system has two very different workloads.

Control plane

Handles:

merchants

users

authentication

menus

orders

payments

analytics

configuration

dashboard

webhooks

audit logs

This can scale normally behind a load balancer.

Voice plane

Handles:

WebSocket audio

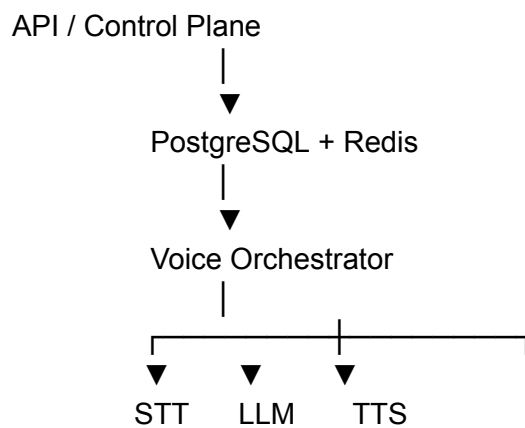
STT streaming

conversation turns

TTS streaming

latency-sensitive state

This needs special treatment.



Do not make the dashboard/API and the audio-processing code a single undifferentiated runtime forever.

---

### 3. Refactor strategy: do NOT start by rewriting everything

Your safest sequence is:

Current MVP



Stabilize



Modularize



Introduce production DB



Introduce Redis



Introduce queue



Add security



Add observability



Containerize



Deploy staging



Load test



Pilot



Scale

Do not simultaneously:

rewrite frontend,

change database,

change AI provider,

change telephony provider,

and introduce Kubernetes.

That creates impossible-to-debug failures.

---

#### 4. Step 1 — Freeze the existing behavior

Before changing architecture, create a behavioral baseline.

Document the existing call flow:

Incoming call

- /voice
- TwiML
- /media-stream
- STT
- processDialogueTurn()
- TTS
- order confirmation
- payment
- notification
- dispatch

Also document browser simulation:

Browser

- /web-stream
- STT
- dialogue
- TTS

Record expected behavior for:

greeting

one item

multiple items

quantity correction

price question

menu question

address

landmark

confirmation

cancellation

repeat order

missed call

DTMF

payment

failed AI service

disconnected call

This becomes your regression suite.

---

5. Step 2 — Turn server.js into a composition root

This is your most important refactor.

Currently server.js owns too many responsibilities.

It should eventually look conceptually like:

```
server/  
  src/  
    app.js  
    bootstrap.js  
  
  config/  
  routes/  
  controllers/  
  middleware/  
  websocket/
```

domain/  
services/  
repositories/  
integrations/  
workers/  
events/  
schemas/  
errors/  
observability/

server.js

Should only:

load config  
→ create app  
→ register middleware  
→ register routes  
→ create WebSocket server  
→ initialize infrastructure  
→ start server

Nothing else.

---

## 6. Step 3 — Introduce domain boundaries

Your current services are a good starting point, but they are not yet proper domain boundaries.

Create:

domain/  
calls/  
conversations/  
orders/  
catalog/  
customers/  
payments/  
notifications/  
merchants/

Each domain should have:



controller  
service  
repository  
schema  
events

Example:

```
orders/  
  order.controller.js  
  order.service.js  
  order.repository.js  
  order.schema.js  
  order.events.js  
  order.state-machine.js
```

---

#### 7. Step 4 — Make the Order State Machine authoritative

This is more important than almost every AI improvement.

Use:

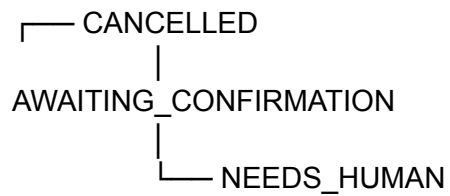
```
NEW  
↓  
COLLECTING_ITEMS  
↓  
COLLECTING_ADDRESS  
↓  
VALIDATING  
↓  
AWAITING_CONFIRMATION  
↓  
CONFIRMED  
↓  
PAYMENT_PENDING  
↓  
PAYMENT_CONFIRMED  
↓  
DISPATCH_PENDING  
↓
```

DISPATCHED



COMPLETED

With failure branches:



The AI can propose:

```
{
  "intent": "add_item",
  "item": "chicken biryani",
  "quantity": 2
}
```

The state machine decides whether that operation is legal.

---

8. Step 5 — Never allow the LLM to determine authoritative price

This is a critical production rule.

Current AI output includes prices and totals.

That should be removed from the authority path.

Bad:

```
LLM
└─ total = 440
```

Correct:

```
LLM
└─ chicken_biryani × 2
```

Catalog DB

└─ ₹220

Order engine

└─ ₹440

The database calculates price.

The LLM never gets to invent:

price

tax

discount

stock

item availability

delivery charge

---

## 9. Step 6 — Replace SQLite with PostgreSQL

Your current database is appropriate for an MVP but not for the production service.

Move to PostgreSQL.

Core schema

tenants

id

name

status

created\_at

users

id

tenant\_id

email  
password\_hash / identity\_provider\_id  
role  
status  
created\_at

## restaurants

id  
tenant\_id  
name  
phone  
timezone  
currency  
status

## branches

id  
restaurant\_id  
name  
phone\_number  
address  
latitude  
longitude  
status

## catalog\_categories

id  
restaurant\_id  
name  
sort\_order  
active

## catalog\_items

id  
category\_id  
sku  
name  
name\_localized  
description  
price

available  
is\_special  
version  
created\_at  
updated\_at

catalog\_item\_variants

id  
item\_id  
name  
price\_delta  
active

catalog\_item\_modifiers

id  
item\_id  
name  
price\_delta  
active

customers

id  
restaurant\_id  
phone  
name  
preferred\_language  
created\_at  
updated\_at

customer\_addresses

id  
customer\_id  
label  
spoken\_address  
formatted\_address  
landmark  
latitude  
longitude  
is\_default

calls

id  
restaurant\_id  
provider  
provider\_call\_id  
customer\_id  
status  
language  
started\_at  
answered\_at  
ended\_at  
duration\_ms

conversations

id  
call\_id  
current\_state  
version  
created\_at  
updated\_at

conversation\_messages

id  
conversation\_id  
role  
text  
language  
confidence  
created\_at

orders

id  
restaurant\_id  
call\_id  
customer\_id  
status  
subtotal  
tax  
discount  
delivery\_fee

total  
currency  
version  
created\_at  
updated\_at

order\_items

id  
order\_id  
catalog\_item\_id  
item\_name\_snapshot  
unit\_price\_snapshot  
quantity  
line\_total

The word snapshot is important.

If a restaurant later changes Chicken Biryani from ₹220 to ₹250, an old order must still say ₹220.

---

## 10. Step 7 — Add strong database constraints

Do not rely solely on application code.

Examples:

UNIQUE(restaurant\_id, sku)

UNIQUE(provider, provider\_call\_id)

CHECK(quantity > 0)

CHECK(total >= 0)

FOREIGN KEY(order\_id)

FOREIGN KEY(customer\_id)

Add indexes:

calls(restaurant\_id, started\_at)  
orders(restaurant\_id, created\_at)  
orders(status)  
customers(restaurant\_id, phone)  
conversation\_messages(conversation\_id, created\_at)  
catalog\_items(category\_id, available)  
payments(order\_id)

---

## 11. Step 8 — Introduce database migrations

Never run raw schema creation on application startup in production.

Replace:

CREATE TABLE IF NOT EXISTS ...

with:

migrations/  
  001\_initial\_schema  
  002\_customer\_addresses  
  003\_order\_state\_machine  
  004\_payment\_events  
  ...

Use one migration tool consistently.

The deployment pipeline must run:

backup  
→ migration  
→ health check  
→ rollout

Never edit production schema manually.

---

## 12. Step 9 — Add transaction boundaries



An order confirmation is a transaction.

Conceptually:

BEGIN

validate order  
create order  
create order\_items  
write audit event  
update conversation state

COMMIT

If any critical step fails:

ROLLBACK

Do not create the order and then discover that the order items failed.

---

### 13. Step 10 — Introduce Redis

Redis should be ephemeral.

Use it for:

active call state

distributed locks

rate limits

short-lived OTPs

WebSocket presence

idempotency keys

queue backend if using BullMQ

Example:

voicecart:call:<callId>

```
{  
  conversationId,  
  currentState,  
  lastActivity,  
  language  
}
```

PostgreSQL remains the durable source of truth.

---

#### 14. Step 11 — Eliminate in-memory sessions

Your current:

```
const sessions = new Map();
```

must disappear from production.

Why?

Instance A:

call → instance A  
session lives in RAM A

Load balancer sends next request to instance B:

session missing

That is a production failure.

Use:

Redis  
+  
PostgreSQL

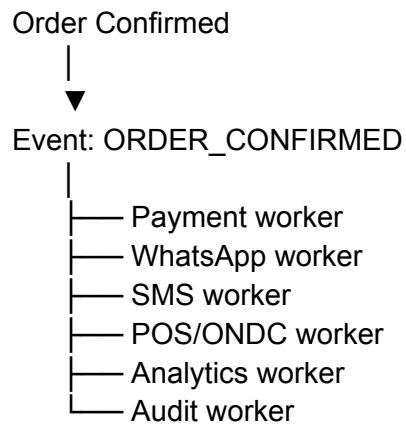
instead.

---

## 15. Step 12 — Introduce an event/queue layer

Do not perform every external operation synchronously during the call.

After order confirmation:



This makes the call path much faster.

---

## 16. Synchronous vs asynchronous operations

Synchronous

Keep these in the live conversation:

STT

LLM intent extraction

catalog lookup

order state update

price validation

confirmation response

TTS

Asynchronous

Move these to workers:

WhatsApp

SMS

email

payment notifications

analytics aggregation

audio archival

POS retry

ONDC dispatch where the protocol allows

reporting

transcription post-processing

---

17. Step 13 — Build an Integration Adapter layer

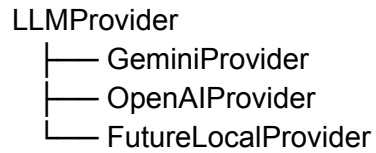
Never let business logic directly call Twilio, Google, WhatsApp, Razorpay, ONDC etc.

Use:

integrations/  
  telephony/  
    stt/  
    llm/  
    tts/  
  payments/

messaging/  
dispatch/  
geocoding/

Example:



The order service only sees:

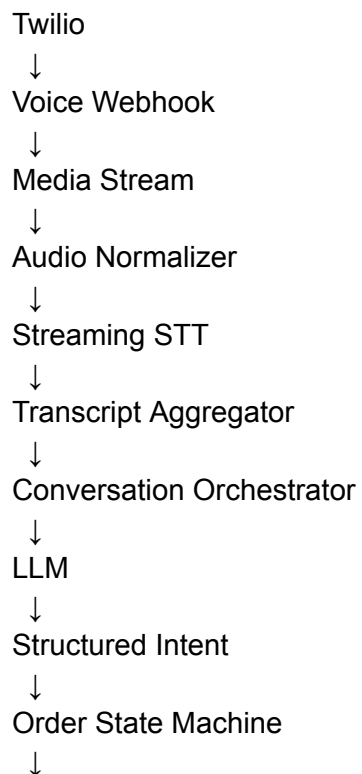
`llm.extractOrderIntent(...)`

It doesn't know which vendor implements it.

---

## 18. Your cloud AI pipeline

Since you explicitly said the current setup is API/cloud, the production pipeline should be:



Catalog Validator



Order Service



TTS



Twilio Audio

Do not add unnecessary RAG/agent frameworks here yet.

---

## 19. STT production design

The STT adapter should return:

```
{
  "text": "two chicken biryani",
  "language": "en-IN",
  "confidence": 0.93,
  "is_final": true,
  "provider_request_id": "..."
}
```

Track:

first\_partial\_ms  
final\_transcript\_ms  
confidence  
language  
provider  
error

Important

Do not run the LLM on every interim transcript.

Use:

interim transcript → UI/logging  
final transcript → dialogue engine

Otherwise cost and latency increase unnecessarily.

---

## 20. LLM production design

Your current dialogueManager.js is doing too much.

Split:

```
conversation/  
  conversation.service.js  
  intent.service.js  
  response.service.js  
  prompt.service.js  
  context.service.js
```

The LLM should output a constrained structure:

```
{  
  "intent": "ADD_ITEM",  
  "entities": {  
    "item_name": "Chicken Biryani",  
    "quantity": 2  
  },  
  "language": "en-IN",  
  "confidence": 0.96  
}
```

Then:

```
Intent  
↓  
Validator  
↓  
Domain operation
```

---

## 21. Never trust raw LLM JSON

Validate it with a schema.

For example:

LLM output

↓

JSON parse

↓

Schema validation

↓

Semantic validation

↓

Domain command

Reject:

```
{  
  "total": 99999999  
}
```

if the LLM tries to inject an invalid amount.

---

## 22. Prompt/version management

Do not bury production prompts inside source code forever.

Use versioned prompt definitions:

```
prompts/  
  order-intent/  
    v1  
    v2  
  confirmation/  
    v1
```

Store:

```
prompt_version  
model  
temperature  
token_usage  
latency  
outcome
```



Then you can compare:

Prompt v3

vs

Prompt v4

without guessing.

---

## 23. TTS production design

The TTS layer should receive:

text

language

voice

speed

and produce:

audio stream

duration

provider request id

Add caching for repeated phrases:

"Vanakkam! Welcome to VoiceCart."

"Would you like anything else?"

"Please confirm your order."

Do not synthesize the same static sentence repeatedly.

---

## 24. Call state and disconnect recovery

You need explicit handling for:

caller disconnects

Twilio disconnects

STT times out  
LLM times out  
TTS fails  
server restarts  
provider webhook retries

For every call:

started  
→ active  
→ completing  
→ completed

with failure states:

failed  
abandoned  
provider\_error  
human\_handoff

---

25. Idempotency is mandatory

Telephony providers retry webhooks.

Payment providers retry webhooks.

Your API must therefore accept duplicate requests safely.

For example:

Idempotency-Key

or a unique provider event ID.

Database:

UNIQUE(provider, provider\_event\_id)

So this:

Payment success webhook

Payment success webhook  
Payment success webhook

results in:

ONE payment state transition

not three.

---

## 26. Webhook security

Your current public webhook endpoints need hardening.

For every telephony/payment webhook:

signature verification  
→ timestamp validation  
→ replay protection  
→ schema validation  
→ idempotency  
→ processing

Never trust:

req.body.From  
req.body.Digits  
req.body.orderId

without validating source and schema.

---

## 27. WebSocket security

You currently have:

/media-stream  
/web-stream  
/dashboard-ws

These should have different security models.

Telephony stream

Authenticated by provider handshake/signature or validated call context.

Dashboard WebSocket

Require:

JWT / session token

tenant\_id

role

Browser voice simulator

Authenticated development/test endpoint only.

Do not expose a completely open /web-stream in production.

---

## 28. Multi-tenancy

This is one of the most important missing pieces.

Every business row should ultimately be associated with:

tenant\_id

or an unambiguous parent that gives tenant ownership.

Example:

tenant

↓

restaurant

↓

branch

↓

menu

↓

order

↓  
call

Every query must scope accordingly.

Bad:

```
SELECT * FROM orders;
```

Good:

```
SELECT *  
FROM orders  
WHERE restaurant_id = $1;
```

The API must never allow a merchant to provide arbitrary `restaurant_id` and access someone else's data.

Derive tenant identity from authenticated context.

---

## 29. Authentication and RBAC

Add:

Identity

↓

Authentication

↓

Tenant Resolution

↓

Authorization

↓

Controller

Roles:

platform\_admin  
restaurant\_owner  
restaurant\_manager  
restaurant\_staff  
support

Permissions should be explicit.

Example:

restaurant\_staff

can: read orders

can: update order status

cannot:

modify billing

manage users

change integrations

---

### 30. API design

Move from ad-hoc endpoints to versioned APIs:

/api/v1/

Examples:

GET /api/v1/restaurants

GET /api/v1/catalog/items

POST /api/v1/catalog/items

GET /api/v1/calls

GET /api/v1/calls/:id

GET /api/v1/orders

POST /api/v1/orders/:id/cancel

GET /api/v1/analytics

Telephony webhooks can be separated:

/webhooks/twilio/voice

/webhooks/twilio/status

/webhooks/payment

/webhooks/ondc

---

### 31. Standard API response format

Don't return arbitrary structures from every route.

For successful responses:

```
{
  "data": {},
  "meta": {}
}
```

For errors:

```
{
  "error": {
    "code": "ORDER_NOT_FOUND",
    "message": "Order not found",
    "request_id": "req_123"
  }
}
```

Never expose raw internal database/provider error messages to customers.

---

### 32. Error handling

Create centralized error classes:

ValidationError  
AuthenticationError  
AuthorizationError  
NotFoundError  
ConflictError  
ProviderError  
RateLimitError  
InfrastructureError

Then one error middleware translates them into HTTP responses.

---

### 33. Configuration management

Separate:

development  
staging  
production

Use environment variables/secrets for:

DATABASE\_URL  
REDIS\_URL  
TWILIO\_\*  
GEMINI\_\*  
PAYMENT\_\*  
WHATSAPP\_\*  
ONDC\_\*  
OBJECT\_STORAGE\_\*  
JWT\_\*

Never put secrets in:

source code

Git

frontend

Docker image

logs.

---

### 34. Remove mock behavior from production

This is a critical issue in the current code.

Today, some integrations deliberately fall back to:

mock STT  
mock TTS



mock WhatsApp  
mock payment  
mock ONDC  
mock callback

That is useful during development but dangerous in production.

Production should behave like:

provider success  
OR  
provider failure  
↓  
explicit failure state

Never:

provider failure  
↓  
pretend success

For example, an ONDC failure must not become a fake successful order.

---

### 35. Replace filesystem call recordings

Current:

./recordings/call\_<id>.wav

is not production-safe.

If two application instances exist:

instance A  
recordings/

instance B  
recordings/

They have different disks.

Use object storage:

Object Storage

/tenant/  
/restaurant/  
/year/  
/month/  
/call-id.wav

Database stores only:

object\_key  
bucket  
duration  
content\_type  
checksum

Generate signed URLs when operators need access.

---

### 36. Privacy and retention

Phone numbers, addresses, calls and recordings are sensitive customer data.

Define retention policies.

Example:

raw call audio → 30/60/90 days  
transcripts → longer  
orders → business retention period  
audit logs → longer

The exact period should reflect your legal/accounting requirements.

Add:

delete customer data  
export customer data  
mask phone numbers in logs  
restrict recording access

Do not log complete phone numbers everywhere.

---

### 37. Geocoding and delivery address flow

Your current geocoding service should be turned into:

spoken address



normalizer



geocoder



confidence



validation

Example:

confidence > threshold

→ accept

confidence medium

→ ask landmark

confidence low

→ send secure pin-drop link

Don't store coordinates by simply appending them to an address string.

Use structured columns:

latitude

longitude

accuracy

source

verified\_at

---

### 38. Payment architecture

Payment must have its own lifecycle:

PAYMENT\_NOT\_REQUIRED  
PAYMENT\_PENDING  
PAYMENT\_LINK\_CREATED  
PAYMENT\_PROCESSING  
PAYMENT\_CONFIRMED  
PAYMENT\_FAILED  
PAYMENT\_EXPIRED  
REFUNDED

Never assume:

payment link created = payment completed

Order status and payment status are separate state machines.

---

### 39. Dispatch architecture

Likewise:

DISPATCH\_PENDING  
DISPATCH\_ACCEPTED  
PREPARING  
READY  
OUT\_FOR\_DELIVERY  
DELIVERED  
FAILED  
CANCELLED

Do not overload:

orders.status

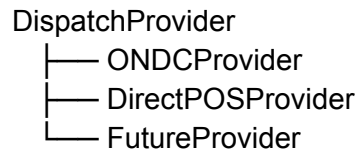
with every operational status.

---

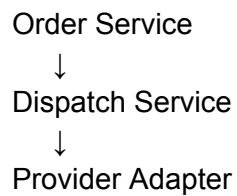
### 40. POS/ONDC adapters

Your application should not know ONDC/POS protocol details.

Create:



Then:



That lets you add restaurant-specific POS integrations later without rewriting order logic.

---

#### 41. Frontend architecture

Your current React application is functional but too centralized.

Move toward:

client/src/

app/

router/

providers/

components/

ui/

features/

dashboard/

calls/

orders/

catalog/

analytics/

merchants/

hooks/

services/

api/

state/

types/

utils/

---

## 42. Frontend API layer

Do not scatter:

`fetch('/api/catalog')`

`fetch('/api/orders')`

`fetch('/api/stats')`

through components.

Create:

api/

client.js

calls.api.js

orders.api.js

catalog.api.js

analytics.api.js

merchants.api.js

Example:

`ordersApi.list()`

`ordersApi.get(id)`

`ordersApi.cancel(id)`

Components should not know URLs.

---

## 43. Custom hooks

This directly addresses your request about hooks.

Create:

hooks/

- useAuth.js
- useCalls.js
- useOrders.js
- useCatalog.js
- useAnalytics.js
- useRealtime.js
- useVoiceSession.js
- usePagination.js
- useDebounce.js

useCalls

Handles:

- fetch calls
- loading
- error
- pagination
- refresh
- filters

useRealtime

Handles:

- connect
- disconnect
- reconnect
- heartbeat
- event subscription

useVoiceSession

Handles:

- WebSocket lifecycle
- audio state
- transcript

assistant response  
session status  
errors

Do not put all of this in App.jsx.

---

#### 44. Frontend state strategy

Separate state into:

Server state

Use a server-state abstraction such as TanStack Query.

orders  
calls  
catalog  
analytics

Client/UI state

Use:

React state  
Context  
or small state store

for:

theme  
selected order  
open modal  
sidebar  
filters

Do not duplicate server state across many components.

---

#### 45. React side effects



A production React rule:

> A component should describe UI. Hooks should own side effects.

So:

Component



Hook



API/service

instead of:

Component



fetch



WebSocket



setInterval



state manipulation

Your current App.jsx is doing polling, WebSocket connection, theme management and dashboard aggregation. Extract those responsibilities.

---

#### 46. Realtime architecture

Do not blindly broadcast every database event to every dashboard client.

Use tenant-scoped channels:

tenant:123

|— calls

|— orders

|— notifications

Authenticated dashboard session subscribes only to its tenant.

Use event types:

CALL\_STARTED  
CALL\_ENDED  
TRANSCRIPT\_UPDATED  
ORDER\_CREATED  
ORDER\_CONFIRMED  
PAYMENT\_UPDATED  
DISPATCH\_UPDATED

---

#### 47. WebSocket reconnect strategy

Client:

connect  
↓  
heartbeat  
↓  
network failure  
↓  
exponential backoff  
↓  
reconnect  
↓  
resynchronize

Important:

WebSocket is not the source of truth.

After reconnect:

REST fetch current state  
+  
WebSocket future events

Otherwise missed events cause stale dashboards.

---

#### 48. Frontend route separation

Structure routes around business functions:

/login

/dashboard

/calls

/calls/:id

/orders

/orders/:id

/catalog

/customers

/analytics

/settings

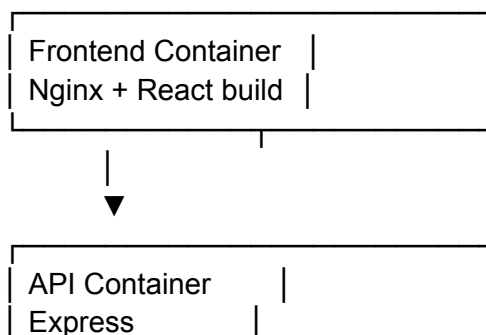
/integrations

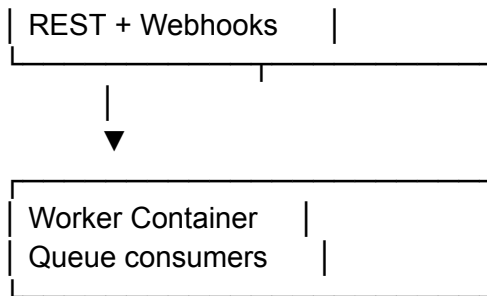
Keep /call as a customer-facing/voice-testing surface if you need it.

---

#### 49. Container architecture

For the initial production version, use four logical containers.





Managed:  
PostgreSQL  
Redis  
Object Storage

For the voice WebSocket workload, you can initially keep it in the API/voice container if it is independently scalable, then split it into a voice gateway when traffic warrants it.

---

50. Why not 15 containers?

Because you're a solo founder.

Running:

API  
Voice  
STT  
LLM  
TTS  
Orders  
Catalog  
Payment  
WhatsApp  
ONDC  
Analytics  
Users

...

as separate microservices immediately creates:

deployment complexity

network failures

service discovery

tracing requirements

version coordination

more CI/CD

more logs

more operational cost

Use logical separation first.

Physical separation comes after traffic justifies it.

---

## 51. Docker image strategy

Each deployable component gets its own Dockerfile.

docker/

api.Dockerfile

worker.Dockerfile

frontend.Dockerfile

Use:

multi-stage builds

non-root user

minimal runtime image

healthcheck

immutable tag

Don't run as root.

Don't bake secrets into the image.

---

## 52. API container startup

Container startup should be:

read config



connect DB



verify Redis



verify required integrations



start application

But do not run database migrations from every replica simultaneously.

Use a separate deployment/migration job.

---

## 53. Health checks

You need:

/health/live

/health/ready

Liveness

"Is the process alive?"

Readiness

"Can this instance safely receive traffic?"

Readiness checks:

DB

Redis

critical configuration

Don't mark the application unhealthy because an optional WhatsApp provider is temporarily down.

---

## 54. Graceful shutdown

When cloud platform sends termination:

stop accepting new calls

↓

stop accepting new HTTP work

↓

finish safe in-flight work

↓

close WebSockets

↓

close queue consumers

↓

close DB connections

↓

exit

Set a termination grace period.

For voice sessions, persist enough state that a new instance can resume or safely recover.

---

## 55. CI/CD pipeline

Every merge should execute:

Git push

↓

Lint

↓

Type/static checks

↓

Unit tests

↓

Integration tests

↓  
Build  
↓  
Docker build  
↓  
Security scan  
↓  
Push image  
↓  
Deploy staging  
↓  
Smoke tests

Production:

manual approval  
↓  
database migration  
↓  
deploy  
↓  
health checks  
↓  
traffic shift  
↓  
monitor

---

## 56. Deployment strategy

Use three environments:

development  
staging  
production

Staging

Must be as production-like as reasonably possible.

Use:



real PostgreSQL

real Redis

test telephony account

test payment environment

test messaging account

production-like containers

Do not use mock providers to qualify a production release.

---

57. Blue/green or canary deployment

Once you have revenue, don't deploy blindly.

Use:

Version A = current

Version B = new

10% traffic → B

50% → B

100% → B

Monitor:

call failures

order failures

latency

payment failures

STT failures

LLM errors

Rollback quickly if thresholds worsen.

---

## 58. Observability

You need three pillars.

### Logs

Structured JSON:

```
{  
  "timestamp": "...",  
  "level": "info",  
  "service": "voice-service",  
  "request_id": "...",  
  "call_id": "...",  
  "tenant_id": "...",  
  "event": "stt_completed",  
  "latency_ms": 412  
}
```

### Metrics

Track:

calls\_started  
calls\_completed  
calls\_abandoned

orders\_created  
orders\_confirmed  
orders\_cancelled

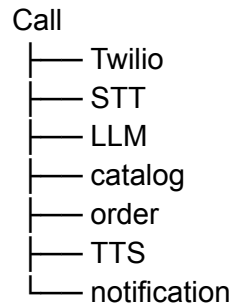
stt\_latency  
llm\_latency  
tts\_latency  
e2e\_latency

payment\_success  
payment\_failure

provider\_errors

### Traces

One call should have one trace:



This is enormously useful when debugging.

---

59. SLOs you should define

Don't say "fast".

Define targets.

For example:

API availability:  $\geq 99.9\%$   
Webhook availability:  $\geq 99.95\%$   
Call setup reliability:  $\geq 99.9\%$   
STT turn latency: target  $< 1s$   
LLM turn latency: target  $< 1.5s$   
TTS first audio: target  $< 1s$

Exact targets should be validated against the actual telephony/AI providers you select.

---

60. AI-specific observability

Track each AI request:

provider  
model  
prompt\_version  
request\_id

call\_id  
language  
input\_tokens  
output\_tokens  
latency  
error  
confidence

Also:

order extraction accuracy  
hallucination rate  
clarification rate  
human handoff rate

The business KPI isn't "LLM latency."

It's:

> What percentage of calls become correct orders without human intervention?

---

## 61. AI evaluation pipeline

Create:

datasets/  
golden/  
regression/  
production\_reviewed/

Every AI change is tested against known calls.

Example:

Input:

"rendu chicken biryani"

Expected:

2 × Chicken Biryani

Test:

STT accuracy  
intent accuracy  
item mapping  
quantity  
language  
final response

---

## 62. Human feedback loop

The merchant dashboard should allow:

Correct transcript  
Correct item  
Correct quantity  
Mark failed call  
Mark wrong AI response

Save that as training/evaluation data.

Then:

production calls  
↓  
human corrections  
↓  
review dataset  
↓  
evaluation  
↓  
prompt/model improvement  
↓  
staging  
↓  
production

This is much more valuable than blindly fine-tuning.

---

### 63. Do not jump to RAG yet

For the restaurant menu, ordinary indexed database lookup is generally better than vector retrieval initially.

Use:

catalog SQL query

for exact menu items and availability.

Use embeddings/RAG later for:

FAQ

restaurant policies

long descriptions

natural-language menu discovery

knowledge articles

Do not introduce a vector database merely because modern AI architecture diagrams contain one.

---

### 64. Do not introduce agent frameworks yet

Your workflow is relatively deterministic:

understand

→ validate

→ update cart

→ ask next question

→ confirm

→ place order

This isn't a good place for autonomous agents.

Use explicit orchestration.

You will get:

better latency

lower cost

easier debugging

easier compliance

predictable behavior

---

65. API call resilience

Every external provider needs:

timeout

retry

exponential backoff

circuit breaker

fallback

idempotency

observability

But there is an important distinction:

Safe retry

GET

read catalog

Usually safe.

Dangerous retry

POST payment

POST order

Requires idempotency.

Never blindly retry order creation.

---

## 66. Provider timeout policies

For live voice calls, don't allow:

LLM hangs 30 seconds

The caller will leave.

Use strict budgets:

STT timeout

LLM timeout

TTS timeout

If the budget is exceeded:

"Sorry, one moment..."

or move to fallback/human handling.

---

## 67. Circuit breakers

Suppose the LLM provider is failing.

Without a circuit breaker:

1000 calls

x

repeated failed requests

With one:

provider failing



↓  
open circuit  
↓  
fail fast / alternate path  
↓  
periodic recovery probe

This protects the system and the provider bill.

---

## 68. Rate limiting

At minimum:

IP  
user  
tenant  
phone number  
API key  
webhook

Have separate limits.

For example:

public API  
dashboard API  
webhooks  
authentication

Don't apply one generic limit to everything.

---

## 69. Security headers

For frontend/API:

HTTPS only

secure cookies if applicable

HSTS

CSP

X-Content-Type-Options

frame restrictions

CORS allowlist

Your current unrestricted:

```
app.use(cors());
```

should become an explicit origin policy.

---

## 70. Input validation

Validate every input:

query params

path params

JSON bodies

webhook payloads

WebSocket messages

LLM outputs

catalog updates

payment callbacks

No req.body should flow directly into a SQL query or business action without schema validation.

---

## 71. Secrets and key rotation

Maintain:

secret management system

with:

Twilio key  
GEMINI key  
payment secret  
database credentials  
storage credentials  
JWT secrets

Rotate keys periodically.

Do not expose cloud credentials to the React app.

---

## 72. Frontend security

The browser gets:

public API configuration

It must never get:

Gemini API key  
Twilio Auth Token  
database password  
payment secret  
ONDC private credentials

All provider calls should happen server-side.

---

## 73. Merchant onboarding

Before selling the product, automate:

Create merchant

↓

Create branch

↓

Assign phone number  
↓  
Upload/import menu  
↓  
Configure timings  
↓  
Configure delivery zone  
↓  
Configure payment  
↓  
Configure WhatsApp  
↓  
Test call  
↓  
Activate

This is a product feature, not merely an engineering task.

---

#### 74. Restaurant configuration model

Do not hardcode:

Anbu Biryani House  
Coimbatore  
₹220

Move all of that into tenant/restaurant configuration.

The runtime should resolve:

call  
↓  
phone number  
↓  
branch  
↓  
restaurant  
↓  
catalog  
↓  
rules

↓  
conversation

This is required for your SaaS model.

---

## 75. Phone-number routing

A production call should be mapped from the incoming number:

Twilio called number

↓  
phone\_number table  
↓  
branch\_id  
↓  
restaurant\_id

Do not hardcode a default merchant.

---

## 76. Call correlation

Every request should carry:

request\_id  
call\_id  
conversation\_id  
tenant\_id  
order\_id

Then a single support case can be traced:

Phone call

- transcript
- AI decision
- order
- payment
- dispatch
- WhatsApp

---

## 77. Audit logging

Every high-value action:

menu price changed  
order confirmed  
order cancelled  
payment confirmed  
customer address updated  
user permission changed

should generate:

audit\_event

With:

actor  
tenant  
action  
entity  
before  
after  
timestamp  
request\_id

---

## 78. Backend repository rules

Repositories should handle persistence.

Services should handle business logic.

Controllers should handle HTTP.

Example:

Controller

↓  
OrderService  
↓  
OrderRepository  
↓  
PostgreSQL

Never:

Controller  
↓  
raw SQL

Your current route handlers contain SQL directly. That should be progressively removed.

---

## 79. API DTOs

Don't expose raw DB rows directly.

Current:

res.json(calls)

should eventually become:

Database row  
↓  
Mapper  
↓  
API DTO  
↓  
JSON response

This protects the database schema from becoming the public API.

---

## 80. Version your API contracts

Use:

/api/v1

Then later:

/api/v2

without breaking installed clients or integrations.

---

## 81. Testing strategy

Use five layers.

Unit tests

Test:

state machine

pricing

validators

catalog matching

prompt parser

adapters

Integration tests

Test:

API

+

Postgres

+

Redis

Contract tests



Test provider interfaces.

End-to-end tests

Simulate:

call

→ STT

→ dialogue

→ order

→ payment

→ dispatch

Load tests

Test:

100 calls

500 calls

1000 concurrent API requests

based on the scale you actually need.

---

82. Production test scenarios

Your golden suite should include:

Normal

One item

Multiple items

Quantity changes

Add/remove

Ambiguity

"one biryani"

"biryani two"

"rendu biriyani"

"one large one"

Language

English

Tamil

Tanglish

Code-switching

Recovery

STT timeout

LLM timeout

TTS error

disconnect

duplicate webhook

payment callback retry

Security

unauthenticated dashboard

cross-tenant access

invalid webhook

forged WebSocket

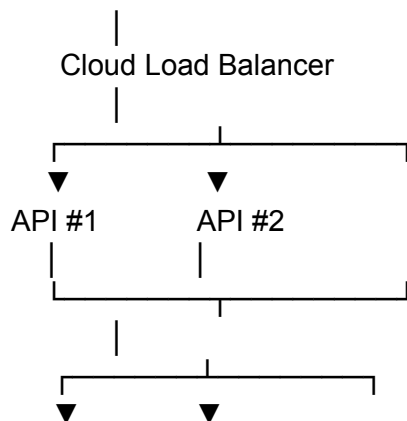
invalid order ID

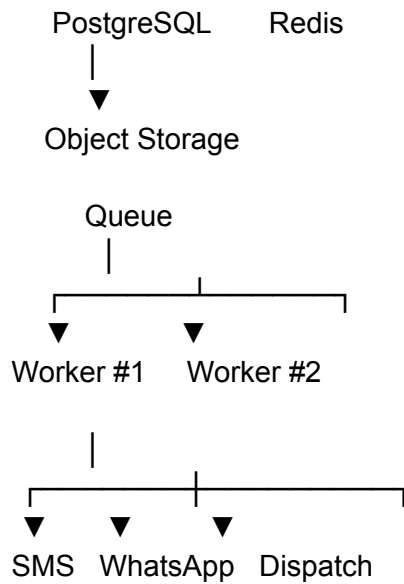
---

### 83. Container deployment topology

A practical first production deployment:

Internet





Voice WebSocket workers can be scaled independently once traffic requires it.

---

#### 84. Horizontal scaling rules

The application should be stateless wherever possible.

That means:

Instance 1  
Instance 2  
Instance 3

must all be equivalent.

No important state in:

RAM  
local disk  
global variables

Store state in:

Postgres  
Redis  
Object storage

---

## 85. WebSocket scaling

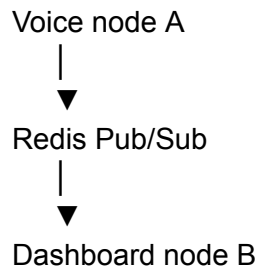
This is the special case.

If multiple voice gateway instances receive related events, you need:

Redis Pub/Sub

or another shared event mechanism.

For example:



Without this, dashboard events remain instance-local.

---

## 86. Database connection pooling

When horizontal scaling:

10 application replicas

×

20 DB connections

=

200 connections

That can overwhelm PostgreSQL.

Use:

bounded pool

+  
connection proxy/pooling if necessary

Tune based on real workload.

---

## 87. Caching strategy

Cache data that changes relatively infrequently:

restaurant config  
menu  
store hours  
FAQ

Use Redis.

But after catalog update:

update DB  
↓  
invalidate cache

Never let cache become the authoritative catalog.

---

## 88. Catalog consistency

A catalog update must propagate safely:

merchant edits price  
↓  
DB transaction  
↓  
catalog\_version++  
↓  
cache invalidation  
↓  
AI context refresh

This matters because your LLM may otherwise be working with stale menu data.

---

## 89. Prompt context optimization

Do not send the entire restaurant database to the LLM every time.

Current dialogue loading of all available catalog rows is acceptable for a prototype but doesn't scale.

Instead:

User utterance



Candidate item lookup



Top relevant catalog items



LLM context

For exact order matching, use normalized database lookup first.

---

## 90. Merchant-specific AI behavior

Store configuration:

voice

language

greeting

tone

upsell\_enabled

max\_silence

delivery\_policy

payment\_options

Then:

Conversation engine



restaurant config



prompt/runtime policy

Do not create a separate code branch for each restaurant.

---

## 91. Analytics model

Don't calculate everything with expensive live SQL.

Separate:

operational tables

from:

analytics events

Capture:

call\_started

call\_answered

intent\_detected

order\_item\_added

order\_confirmed

payment\_success

call\_abandoned

Then aggregate:

daily\_call\_metrics

daily\_order\_metrics

as volume increases.

---

## 92. What the merchant dashboard should actually show

Not just technical data.

Primary metrics:

Calls today  
Orders today  
Conversion %  
Revenue  
Average order value  
Missed calls  
Recovered orders  
Average call duration  
Payment success  
AI handoff rate

Later:

Revenue recovered  
Estimated aggregator commission avoided  
Repeat customer rate  
Peak order time

---

### 93. Operational support screen

You need an internal support view.

For each call:

CALL

↓

audio

↓

transcript

↓

AI decisions

↓

state transitions

↓

order

↓

payment

↓



dispatch

That makes customer support possible.

---

#### 94. Feature flags

Add a feature flag system for:

AI model  
prompt version  
language  
upsell  
repeat-order  
pin-drop  
ONDC  
payment method

Then activate gradually.

Example:

merchant A → new prompt  
merchant B → old prompt

without redeploying.

---

#### 95. Billing and metering

Even before monetization, record:

AI seconds  
call minutes  
STT seconds  
TTS characters  
LLM tokens  
orders  
messages

Then calculate:

cost per call  
cost per successful order  
cost per merchant

This is essential for deciding pricing.

---

96. The business metric that matters most

Build this metric:

cost per successful order

For example:

STT ₹x  
LLM ₹y  
TTS ₹z  
telephony ₹a  
messaging ₹b  
infra ₹c

-----

total order cost

Then compare against:

subscription revenue  
+  
transaction revenue

That gives you unit economics.

---

97. Production logging rule

Do not log:

- full phone number
- full payment data
- full address
- secrets
- tokens
- provider credentials

Use masked values where possible.

Example:

+91\*\*\*\*\*4321

---

## 98. Disaster recovery

Define:

- RPO
- RTO

At minimum:

- Postgres backup
- object storage versioning
- configuration backup
- migration scripts
- deployment history

You should be able to rebuild production from code + backups.

---

## 99. Incident response

Have a runbook for:

- Twilio outage
- LLM outage
- STT outage
- payment outage

database outage  
Redis outage  
queue backlog  
bad deployment

Example:

LLM unavailable  
↓  
enable fallback model/provider  
↓  
monitor  
↓  
disable feature flag

Don't discover your recovery plan during an outage.

---

## 100. Production readiness checklist

### Architecture

- ☐ Modular backend
- ☐ Domain boundaries
- ☐ Stateless API
- ☐ Redis session state
- ☐ PostgreSQL
- ☐ Object storage
- ☐ Queue workers
- ☐ Integration adapters

### Voice

- ☐ Telephony signature verification

- ☐ Streaming STT
- ☐ final/interim separation
- ☐ LLM timeout
- ☐ TTS timeout
- ☐ call recovery
- ☐ disconnect handling
- ☐ call correlation IDs

## AI

- ☐ Structured output
- ☐ schema validation
- ☐ deterministic business rules
- ☐ prompt versioning
- ☐ model versioning
- ☐ evaluation dataset
- ☐ production feedback loop
- ☐ cost monitoring

## Database

- ☐ PostgreSQL
- ☐ migrations
- ☐ transactions
- ☐ constraints

☐ indexes

☐ tenant isolation

☐ backups

☐ connection pooling

## API

☐ /api/v1

☐ authentication

☐ RBAC

☐ rate limiting

☐ validation

☐ idempotency

☐ structured errors

☐ CORS policy

## Frontend

☐ API abstraction

☐ server-state management

☐ custom hooks

☐ realtime hook

☐ auth context

☐ tenant-aware routing

☐ error/loading states

☐ reconnect/resync

## Security

☐ HTTPS

☐ webhook verification

☐ secrets manager

☐ least privilege

☐ tenant isolation

☐ audit logs

☐ PII controls

☐ dependency scanning

## Containers

☐ multi-stage Docker builds

☐ non-root user

☐ health checks

☐ graceful shutdown

☐ immutable images

☐ vulnerability scanning

## Deployment

☐ dev

☐ staging

- ☐ production
- ☐ CI/CD
- ☐ migrations
- ☐ smoke tests
- ☐ rollback
- ☐ canary/blue-green when needed

#### Observability

- ☐ structured logs
- ☐ metrics
- ☐ tracing
- ☐ alerts
- ☐ latency dashboards
- ☐ cost dashboards
- ☐ provider health dashboards

---

101. The exact refactoring order I recommend

This is the sequence I would actually follow on your repository.

#### Sprint 1 — Stabilize

1. Freeze current behavior
2. Establish regression tests
3. Clean configuration
4. Remove hardcoded merchant assumptions
5. Standardize errors



6. Add request/call IDs

## Sprint 2 — Database

7. Design PostgreSQL schema

8. Create migration system

9. Build repositories

10. Add tenant model

11. Add constraints/indexes

12. Add transactions

## Sprint 3 — Backend architecture

13. Thin server.js

14. Introduce controllers

15. Introduce services

16. Introduce repositories

17. Introduce validators

18. Introduce integration adapters

## Sprint 4 — Voice engine

19. Extract media gateway

20. Extract conversation orchestrator

21. Introduce Redis sessions

22. Introduce order state machine

23. Add call recovery

24. Add WebSocket authentication

## Sprint 5 — External services

25. Harden STT

26. Harden LLM

27. Harden TTS

28. Payment lifecycle

29. Messaging worker

30. Dispatch adapter

31. Geocoding workflow

## Sprint 6 — Async infrastructure

32. Queue

33. Workers

34. Retry policy

- 35. Dead-letter queue
- 36. Idempotency
- 37. Outbox/event pattern

#### Sprint 7 — Frontend

- 38. API client
- 39. Auth
- 40. server-state hooks
- 41. realtime hook
- 42. feature modules
- 43. error boundaries
- 44. tenant-aware dashboard

#### Sprint 8 — Security

- 45. RBAC
- 46. rate limits
- 47. webhook signatures
- 48. secret management
- 49. PII controls
- 50. audit logging

#### Sprint 9 — Infrastructure

- 51. Docker
- 52. staging
- 53. production
- 54. health checks
- 55. autoscaling
- 56. object storage
- 57. database backups

#### Sprint 10 — Observability

- 58. logs
- 59. metrics
- 60. tracing
- 61. alerts
- 62. AI evaluation
- 63. cost monitoring

#### Sprint 11 — Production testing

- 64. integration suite
- 65. end-to-end calls
- 66. failure injection
- 67. load tests
- 68. security tests
- 69. disaster recovery test

#### Sprint 12 — Pilot

- 70. one real restaurant
- 71. monitor every call
- 72. fix failure modes
- 73. add second restaurant
- 74. stabilize
- 75. then scale

---

#### 102. What I would NOT change yet

There are several things from our earlier discussions that should remain postponed.

Don't introduce yet

MCP

Agent frameworks

Complex vector DB

Multi-agent orchestration

Speech-to-speech

Kubernetes

Large microservice fleet

Full LLM fine-tuning

They may become useful later, but none solves your current primary problem.

Your primary problem is:

> Can a real customer make a call and receive a correct order every time?

---

### 103. Your final production repository

I would aim for this structure:

```
voicecart/
├── apps/
│   ├── api/
│   ├── worker/
│   └── web/
├── packages/
│   ├── domain/
│   ├── schemas/
│   ├── config/
│   ├── logging/
│   └── integrations/
├── database/
│   ├── migrations/
│   ├── seeds/
│   └── fixtures/
├── infrastructure/
│   ├── docker/
│   ├── deployment/
│   ├── monitoring/
│   └── scripts/
├── tests/
│   ├── unit/
│   ├── integration/
│   ├── e2e/
│   ├── load/
│   └── security/
├── docs/
│   ├── architecture/
│   ├── api/
│   ├── runbooks/
│   └── decisions/
└── .github/
```

```

|   └─ workflows/
|
|   └─ docker-compose.ci.yml
|   └─ package.json
|   └─ README.md

```

This is substantially more maintainable than keeping all backend responsibilities in `server/server.js`.

---

104. One architectural correction I strongly recommend

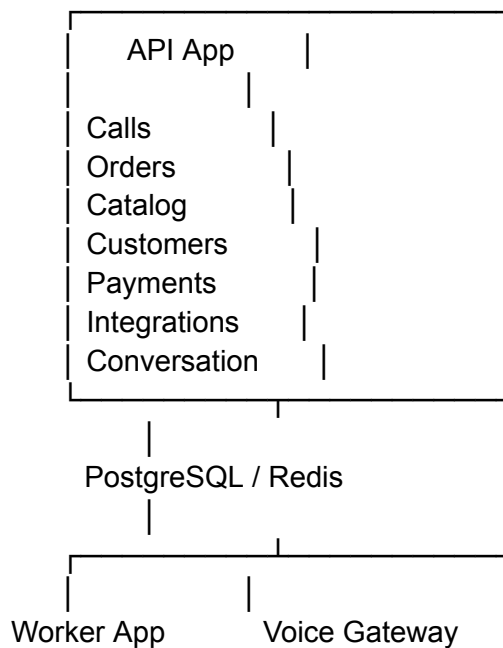
I would not literally implement:

12 microservices

as the first production refactor.

Instead:

Production System



That's your best solo-founder architecture.

Internally, keep strong module boundaries.

Later, when one module becomes the bottleneck:

Conversation module



extract service

or:

Voice gateway



scale independently

This is evolutionary architecture, rather than premature microservices.

---

## 105. The definition of “production ready” for VoiceCart

I would only call your platform production-ready when it can satisfy this scenario:

Customer calls



correct restaurant selected



voice stream established



speech recognized



intent extracted



menu validated



price calculated by backend



order state updated



customer confirms



order transaction committed



payment lifecycle tracked

↓  
dispatch requested  
↓  
WhatsApp/SMS delivered  
↓  
merchant dashboard updates  
↓  
audio/transcript auditable

And then intentionally break things:

LLM down  
STT down  
TTS down  
Redis down  
worker down  
duplicate webhook  
duplicate payment callback  
database restart  
API restart  
browser disconnect  
customer disconnect

The system should fail safely rather than pretend success.

That is the difference between an impressive MVP and a production business.

---

The implementation priority for your existing codebase

I would make these the first ten actual code changes:

1. Extract server.js into app, routes, controllers, websocket, and services.
2. Replace SQLite with PostgreSQL + migrations.
3. Introduce tenant → restaurant → branch ownership.
4. Replace Map() sessions with Redis.

5. Build the deterministic order state machine.
6. Move all pricing/order validation outside the LLM.
7. Introduce provider adapters for STT/LLM/TTS/payment/notifications/dispatch.
8. Add a queue + worker for non-real-time side effects.
9. Add authentication, RBAC, webhook verification, idempotency, and rate limiting.
10. Containerize API/worker/frontend and deploy a staging environment before production.

After those ten, the codebase will have moved from MVP architecture to a credible production foundation. Then you harden observability, load, frontend state management, AI evaluation, disaster recovery, and merchant operations.

The correct goal is not a codebase that is merely “full-grade.” It is a system in which every failure has an explicit behavior, every important state is durable, every external call is observable and retry-safe, every tenant is isolated, and every deployment can be rolled back.